

Cours de Programmation en C – EI-2I

Travaux Dirigés

TD4

Exercice 1 – factorielle

1. Écrire une fonction permettant de calculer la factorielle d'un nombre, en utilisant la formule

$$n! = \begin{cases} \prod_{k=1}^n k & \text{si } n > 0 \\ 1 & \text{si } n=0 \end{cases}$$

2. Ré-écrire cette fonction en utilisant la formule

$$n! = n(n-1)! \quad \text{et} \quad 0! = 1$$

Exercice 2 – Passage par adresse, par valeur

On considère la fonction f définie par

$$f(x) = \sqrt{(x-1)(x-2)}$$

Évidemment, f n'est pas défini pour tout $x \in \mathbb{R}$.

Écrire une fonction `f` ayant comme paramètres un `float x` et un booléen `ok` et qui renvoie la valeur $f(x)$. Par l'intermédiaire de la variable `ok`, la fonction indique si f est définie au point x (`ok` prendra la valeur 1 si la fonction est définie sur x et la valeur 0 si non; dans ce cas, la valeur retournée par la fonction `f` n'a pas d'importance).

Proposez un exemple d'utilisation.

Exercice 3 – Portée des variables

On considère le programme suivant :

```
#include <stdio.h>

int a = 27;

int chose(int a);
int machin();

int chose(int a) {
    return a+17+machin();
}

int machin() {
    return a;
}

int main() {
    int a = 1;
    a = chose(a);
    printf("%d\n", a);
    return 0;
}
```

Quelle est la valeur affichée par ce programme ?

Exercice 4 – Puissance $k^{\text{ème}}$ récursive

Pour calculer x^k (x réel, k entier positif), on peut utiliser le principe de récursion suivant :

- si $k = 0$, alors le résultat est 1.
 - si $k = 2 \times p$ (k est pair), alors on calcule (selon le même principe) $m = x^p$, puis le résultat est $m \times m$.
 - si $k = 2 \times p + 1$ (k est impair), alors on calcule (selon le même principe) $m = x^p$, puis le résultat est $x \times m \times m$.
1. Écrire la fonction récursive `puiss` qui prend en entrée x et k et qui calcule x^k selon le principe décrit ci-dessus.
 2. Dans le cas du calcul de x^{18} , combien d'appels seront fait à la fonction `puiss`
 3. Dans le calcul de x^k avec $k = 2p$, combien d'appels seront fait à la fonction `puiss` (la réponse dépend bien sûr de p).
 4. Ré-écrire la fonction `puiss` sans utiliser la récursivité (mais en utilisant un principe similaire, basé sur l'écriture binaire de k).

Exercice 5 – Pointeurs en folie

On considère le programme C suivant:

```
#include <stdlib.h>
#include <stdio.h>

int main ( )
{
    int a = 1, b = 2, c = 3;
    int *p1, *p2;
    p1 = &a;
    p2 = &c;
    *p1 = ( *p2 )++;
    p1 = p2;
    p2 = &b;
    *p1 -= *p2;
    ++( *p2 );
    *p1 *= *p2;
    a = ( ++( *p2 ) ) * *p1;
    p1 = &a;
    *p2 = *p1 /= *p2 ;

    return EXIT_SUCCESS;
}
```

Remplissez le tableau suivant :

	a	b	c	p1	p2
Init					
<code>p1 = &a;</code>					
<code>p2 = &c;</code>					
<code>*p1 = (*p2)++;</code>					
<code>p1 = p2;</code>					
<code>p2 = &b;</code>					
<code>*p1 -= *p2;</code>					
<code>++(*p2);</code>					
<code>*p1 *= *p2;</code>					
<code>a = (++(*p2)) * *p1;</code>					
<code>p1 = &a;</code>					
<code>*p2 = *p1 /= *p2;</code>					